

데이터 어노테이션 프로그램 가이드

목차

1. 개요

2. 사용 환경 설정

2-1. Window 환경 상세 가이드

2-2. Linux 환경 상세 가이드

3. 데이터 어노테이션 워크플로우

3-1. 데이터 어노테이션 워크플로우 Overview

3-2. 도커 이미지 빌드

3-3. 프로그램 실행

3-4. 이미지 업로드 (Input)

3-5. 1차 자동 세그멘테이션 (Segment)

3-6. 2차 세그멘테이션 수정 (GUI)

3-7. 데이터 내보내기 (Export)

3-8. 학습 (Training)

3-9. 기타 기능

1. 개요: 모델 및 성능

모델

- 현재 1차 자동 마스킹 단계에서 **YOLO26l-seg** 모델 사용
- 학습 데이터는 3가지 배경 환경(도화지 + 바닥 + 책상)으로 구성

학습 데이터

- 총 2000장(클래스별 200장) 구축 후 사용

성능

학습 데이터 규모 (총 이미지 수)	학습 데이터 환경	Mask mAP50-95 테스트 데이터 환경		모델 버전	ROS FPS	YOLO FPS
		흰 도화지	도화지 + 바닥 + 책상			
2000 [Class 별 200]	흰 도화지	96.3	89.3	YOLO26l-seg	36.41 ± 0.63	43.49 ± 0.79
	도화지 + 바닥 + 책상	96.2	96.6			

- 지난 미팅에서 배경(학습 환경)에 따라 성능이 조금씩 달라질 수 있음을 확인
 - 단일 배경보다 다양한 배경으로 학습 시 일반화 성능 향상
 - 배경 합성, 데이터 다양성 확대 등을 시도해서 추가 개선 시도

기타 사항

- 현재 어노테이션 프로그램은 기능적으로 완성된 상태, 추가적인 소스코드 수정 없이 사용 가능
- 모델 성능 개선 시 소스코드 변경 없이 **가중치 파일 교체만으로 업데이트 가능** (추후 모델 성능 개선 시 가중치 파일만 업데이트 예정)

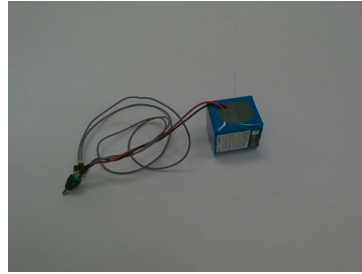
1. 개요: 학습 데이터 클래스 구성



[0]



[1]



[2]



[3]



[4]



[5]



[6]



[7]



[8]



[9]

설명

- 총 10개 클래스([0] ~ [9])로 구성되며, 각 클래스별 대표 객체 예시
- 각 클래스는 특정 객체 유형을 의미, 동일 클래스 내 다양한 환경(배경, 위치, 각도)을 포함하도록 학습 데이터를 구성

1. 개요: 사용 환경 설정 - 환경 세팅 요약 (Windows)

[Windows 환경 세팅 상세 가이드]

Windows

설치 항목

- Nvidia Driver, WSL2, Docker Desktop
- 위 항목이 모두 설치되어 있다면 새로 설치할 필요 없음. 도커 빌드로 이동 [\[docker build\]](#)

모든 명령어(빨간색으로 표시)는 Windows의 경우 Windows PowerShell에 리눅스의 경우 Terminal에 입력

설치 순서

1. NVIDIA Driver 설치

<https://www.nvidia.com/ko-kr/drivers/>

- 해당 링크에서 본인 컴퓨터에 맞는 그래픽카드, 운영체제 선택 후 Geforce Game Ready 드라이버 다운로드

2. WSL2 설치 및 버전 선택

wsl --install

wsl --set-default-version 2

wsl --update

3. Docker Desktop 설치

<https://docs.docker.com/desktop/setup/install/windows-install/>

- 위 링크에서 설치 후 실행
- Settings -> Resources -> WSL Intergration -> Enable integration with my default WSL distro 체크

4. 최종 확인

nvidia-smi

wsl -l -v

wsl --status

docker --version

docker compose version

docker ps

docker run --rm --gpus all nvidia/cuda:12.6.2-base-ubuntu22.04 nvidia-smi

- 모두 정상 출력되면 완료

1. 개요: 사용 환경 설정 - 환경 세팅 요약 (Linux)

[Linux 환경 세팅 상세 가이드]

Linux

설치 항목

- Nvidia Driver, Docker, Nvidia Container Toolkit
- 위 항목이 모두 설치되어 있다면 새로 설치할 필요 없음. 도커 빌드로 이동 [\[docker build\]](#)

모든 명령어(빨간색으로 표시)는 Windows의 경우 Windows PowerShell에 리눅스의 경우 Terminal에 입력

설치 순서

1. NVIDIA Driver 설치

권장 드라이버 확인

`ubuntu-drivers devices`

권장 드라이버 설치

`sudo apt install nvidia-driver-570-open`

`sudo reboot`

2. Docker 설치 및 실행

`sudo apt update`

`sudo apt install docker-ce docker-ce-cli containerd.io`

`docker-buildx-plugin docker-compose-plugin`

`sudo systemctl start docker`

`sudo systemctl enable docker`

3. NVIDIA Container Toolkit 설치

`sudo apt install -y nvidia-container-toolkit`

`sudo nvidia-ctl runtime configure --runtime=docker`

`sudo systemctl restart docker`

- GPU를 Docker 내부에서 사용하기 위해 필요

4. 최종 확인

`nvidia-smi`

`docker --version`

`docker compose version`

`docker ps`

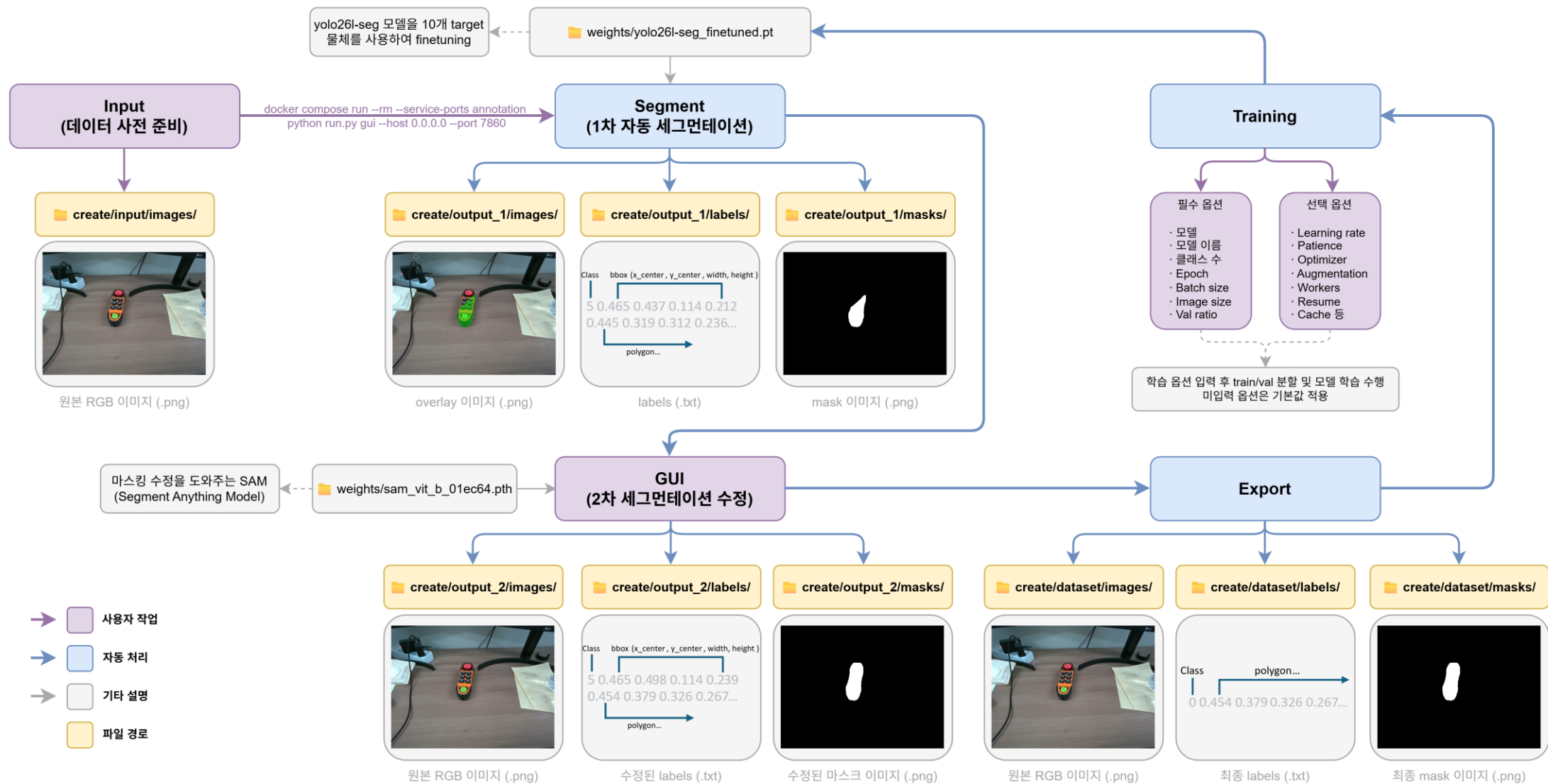
`docker run --rm --gpus all nvidia/cuda:12.6.2-base-ubuntu22.04 nvidia-smi`

- 모두 정상 출력되면 완료

1. 개요: 데이터 어노테이션 워크플로우 - Overview

YOLO26l-seg 기반 Image Segmentation 및 마스크 수정 프로그램

- 640×480 png 이미지로 자동 segmentation 및 수동 수정 후 최종 데이터셋 생성



2. 사용환경설정

2-1. Windows 환경 세팅 상세 가이드

2-1. Window 환경 - Nvidia Driver 설치

모든 명령어(빨간색으로 표시)는 Windows의 경우 Windows PowerShell에 입력

A-1. Windows PowerShell 에 입력

`nvidia-smi`

아래 이미지처럼 이미 드라이버가 존재하면(드라이버 버전이 뜨면) Driver 설치과정 생략. 그렇지 않은 경우 -> [A-2](#). 다음페이지

```
PS C:\Users\user> nvidia-smi
Thu Apr 23 09:14:04 2026
```

NVIDIA-SMI 591.86			Driver Version: 591.86		CUDA Version: 13.1	
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A	Volatile Uncorr. ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util Compute M.
						MIG M.
0	NVIDIA GeForce RTX 5090		WDDM	00000000:02:00.0	On	N/A
0%	49C	P5	26W / 575W	1431MiB / 32607MiB	0%	Default N/A

2-1. Window 환경 - Nvidia Driver 설치

A-2. 링크 접속

<https://www.nvidia.com/ko-kr/drivers/>

A-3. 사용자 컴퓨터에 맞는 그래픽카드, 운영체제 선택 후 찾기 선택

수동 드라이버 검색

제품, 제품 유형 또는 시리즈로 검색

GeForce

GeForce RTX 30 Series

GeForce RTX 3070

Windows 11

Korean

찾기

A-4. 최신 GeForce Game Ready 드라이버 선택

- 보기 선택 -> 다운로드

Windows 10 64-bit Windows 11

GeForce Game Ready 드라이버

Game Ready 드라이버란 무엇인가요?

최신 게임, 패치, DLC에 대한 초고속 지원을 우선시하는 게이머라면 Game Ready 드라이버를 선택하세요.

드라이버 버전:
596.21

출시일:
2026-4-16

파일 크기:
958.86 MB

정보:
Game Ready WHQL

보기

다운로드

드라이버 버전: 596.21 | WHQL
출시일: Thu Apr 16, 2026
운영 체제: Windows 10 64-bit, Windows 11
언어: Korean
파일 크기: 958.86 MB

*이 다운로드에는 NVIDIA 그래픽 드라이버와 GeForce Experience 애플리케이션을 추가로 설치할 수 있는 옵션이 포함되어 있습니다. 소프트웨어 사용에 대한 세부 정보는 각각 NVIDIA GeForce 소프트웨어 라이선스와 GeForce Experience 소프트웨어 라이선스에서 확인할 수 있습니다.

2-1. Window 환경 - Nvidia Driver 설치

A-5. 다운로드 된 .exe 파일 실행 후 다운로드 진행



596.21-desktop-
win10-win11-64
bit-international-
dch-whql

A-6. 재부팅

A-7. 최종 설치 확인

nvidia-smi

```
PS C:\> nvidia-smi
Mon Apr 27 10:26:06 2026
```

NVIDIA-SMI 591.86			Driver Version: 591.86			CUDA Version: 13.1		
GPU	Name	Perf	Driver-Model	Bus-Id	Disp.A	Volatile	Uncorr.	ECC
Fan	Temp		Pwr:Usage/Cap		Memory-Usage	GPU-Util	Compute	M.
							MIG	M.
0	NVIDIA GeForce RTX 5090		WDDM	00000000:02:00.0	On			N/A
0%	47C	P8	24W / 575W	1356MiB / 32607MiB		1%	Default	N/A

정상 출력 확인

2-1. Window 환경 - WSL2 설치

[WSL (Windows Subsystem for Linux) 은 기능 설치이기 때문에 설치 명령어 입력 시 기본 경로는 영향이 없음]

공식 설명서 - <https://learn.microsoft.com/ko-kr/windows/wsl/install>

B-1. WSL 설치

`wsl --install` (해당 명령어는 가장 최신 Linux를 같이 설치함)

`wsl --install -d Ubuntu-22.04` (특정 버전을 설치하고 싶은 경우 ex. 22.04)

B-2. 사용자명 및 비밀번호 입력

* 성공 시 WSL 터미널에 진입하면서 경로와 색깔이 바뀜

B-3. 다시 Windows PowerShell로 복귀

Ctrl + D

```
PS C:\> wsl --install
다운로드 중: Ubuntu
설치 중: Ubuntu
배치가 설치되었습니다. 'wsl.exe -d Ubuntu'을(를) 통해 시작할 수 있습니다.
Ubuntu 시작하는 중...
Provisioning the new WSL instance Ubuntu
This might take a while...
Create a default Unix user account: user
New password:
Retype new password:
passwd: password updated successfully
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

user@DESKTOP-S81END5:/mnt/c$
exit
PS C:\>
```

2-1. Window 환경 - WSL2 설치

[WSL은 기능 설치이기 때문에 설치 명령어 입력 시 기본 경로는 영향이 없음]

공식 설명서 - <https://learn.microsoft.com/ko-kr/windows/wsl/install>

WSL 1, 2 주요 차이

구분	WSL1	WSL2
구조	리눅스 에뮬레이션	실제 리눅스 커널
속도	느림 (I/O 제한)	빠름 (native 수준)
기능	Docker / GPU 불가	Docker / GPU 가능

B-4. 기본으로 WSL2를 사용하도록 지정

`wsl --set-default-version 2`

B-5. WSL Update

`wsl --update`

B-6. WSL2 실행

`wsl`

```
PS C:\> wsl --set-default-version 2
WSL 2와의 주요 차이점에 대한 자세한 내용은 https://aka.ms/wsl2를 참조하세요
작업을 완료했습니다.
PS C:\> wsl --update
업데이트 확인 중입니다.
Linux용 Windows 하위 시스템 최신 버전이 이미 설치되어 있습니다.
PS C:\> wsl
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

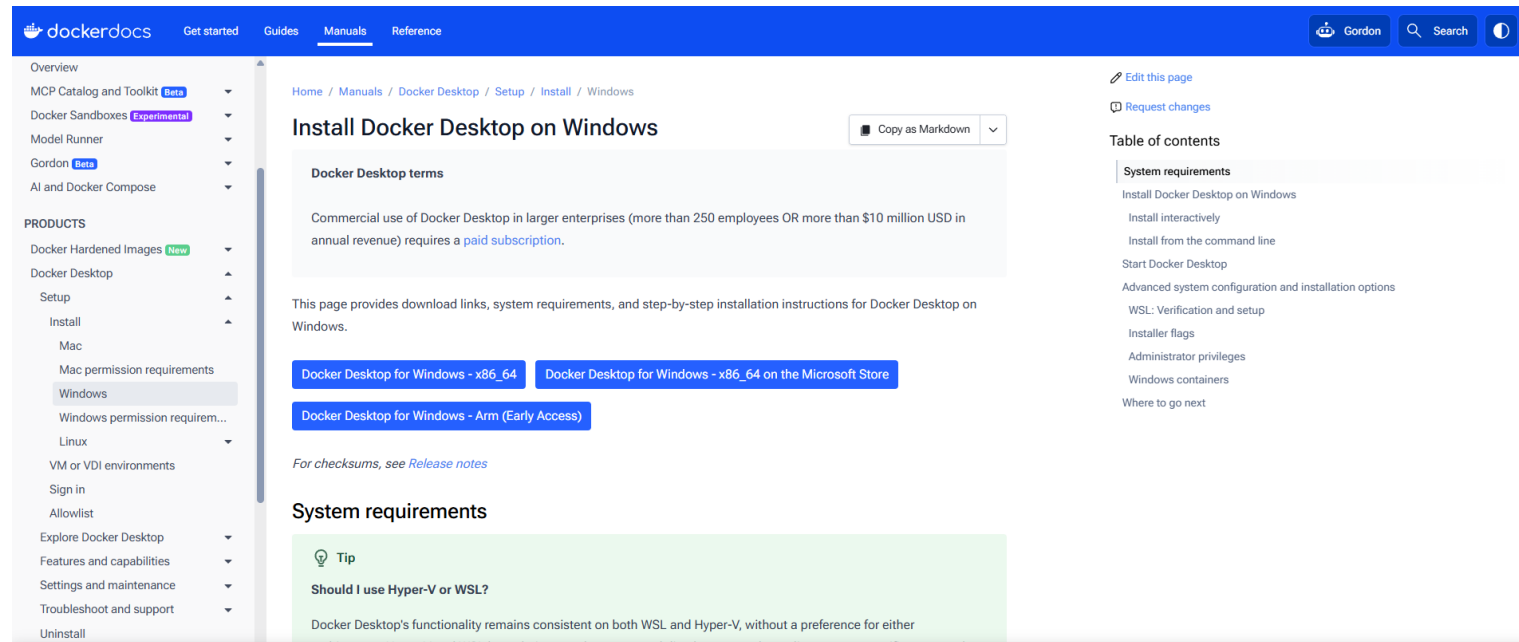
user@DESKTOP-S81END5:/mnt/c$ |
```

2-1. Window 환경 - Docker Desktop 설치

[Docker Desktop은 WSL2 기반이므로 앞선 설치 및 버전 설정을 반드시 진행하고 설치해야함]

C-1. 공식 설치 링크 진입

공식 링크 - <https://docs.docker.com/desktop/setup/install/windows-install/>



C-2. 본인 환경에 맞는 버전으로 설치

- 일반적인 Intel / AMD CPU PC
- ARM 아키텍처 CPU PC

Docker Desktop for Windows - x86_64

Docker Desktop for Windows - x86_64 on the Microsoft Store

Docker Desktop for Windows - Arm (Early Access)

2-1. Window 환경 - Docker Desktop 설치

C-3. 설치 파일 실행 및 다운로드 완료

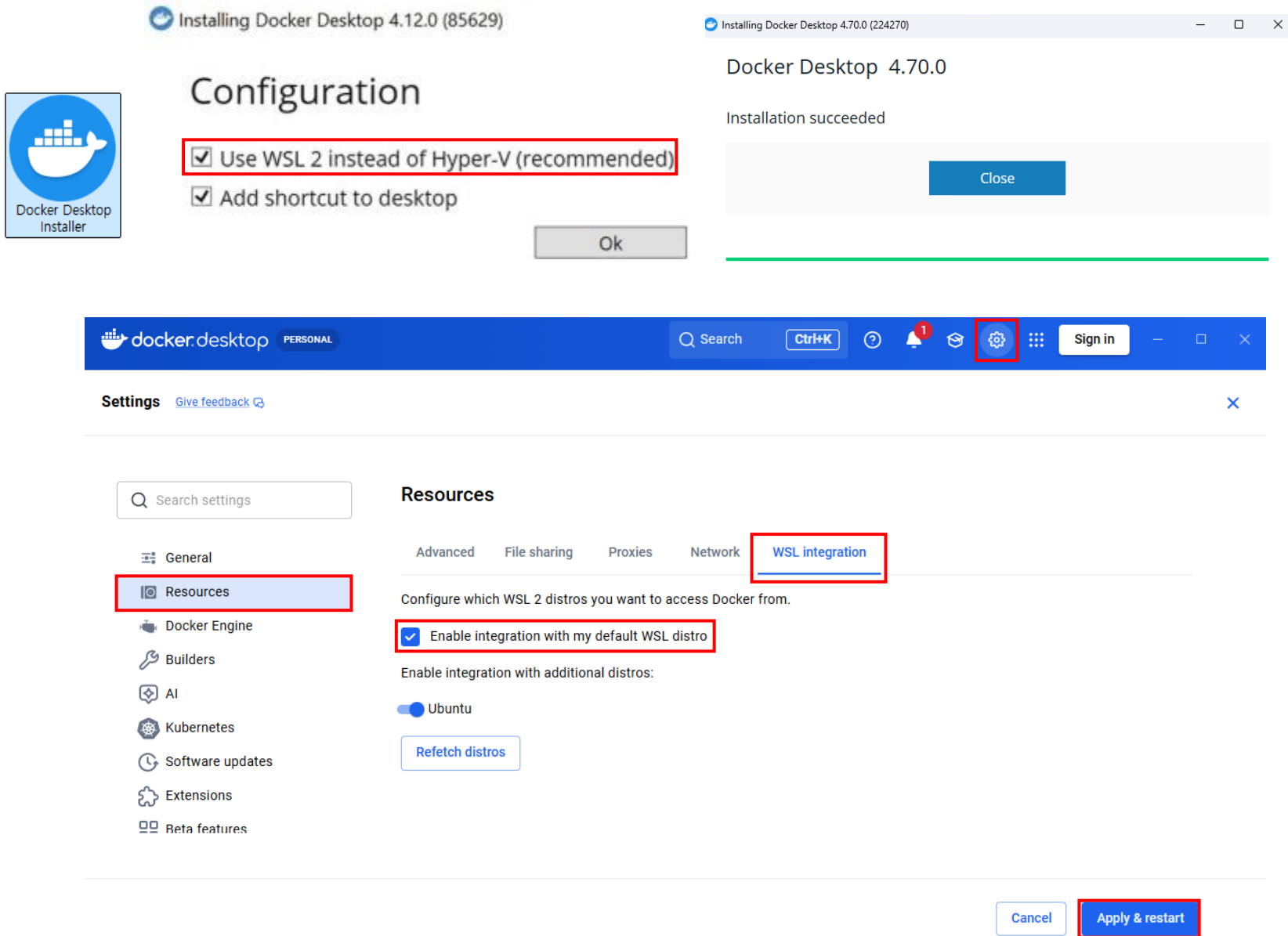
- 다운로드 시 Use WSL2 Instead of Hyper-V 선택
- 뜨지 않으면 기본적으로 Use WSL2로 설치됨



C-4. 실행 (로그인은 선택, 필수 아님)

C-5. WSL Integration 설정

- Settings -> Resources -> WSL Intergration
- Enable integration with my default WSL distro 체크
- Ubuntu 체크 (필수 아님, 추후 개발 편의성을 위함)
- Apply & restart



2-1. Window 환경 - 최종 테스트

D-1. 최종 확인 명령어

nvidia-smi

ws1 -l -v

```
wsl --status
```

```
docker --version
```

docker compose version

```
docker ps
```

```
docker run --rm --gpus all nvidia/cuda:12.6.2-
```

base-ubuntu22.04 nvidia-smi

```
Windows PowerShell
PS C:\> nvidia-smi
Mon Apr 27 10:28:36 2026

+-----+
| NVIDIA-SMI 591.86                 Driver Version: 591.86          CUDA Version: 13.1     |
+-----+-----+
| GPU   Name                               Driver-Model  Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp   Perf         Pwr:Usage/Cap       Memory-Usage  GPU-Util  Compute M.  |
|                               MIG M.               |
+-----+-----+
|  0  NVIDIA GeForce RTX 5090      WDDM        00000000:02:00.0  On          N/A          |
| 0%   47C    P8               28W / 575W        1473MiB / 32607MiB      0%         Default    |
|                               575W                    |
+-----+-----+

PS C:\> wsl -l -v
  NAME                STATE          VERSION
* Ubuntu              Running        2
  docker-desktop      Running        2

PS C:\> wsl --status
기본 배포: Ubuntu
기본 버전: 2

PS C:\> docker --version
Docker version 29.4.0, build 9d7ad9f

PS C:\> docker compose version
Docker Compose version v5.1.2

PS C:\> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\> docker run --rm --gpus all nvidia/cuda:12.6.2-base-ubuntu22.04 nvidia-smi
Mon Apr 27 01:28:37 2026

+-----+
| NVIDIA-SMI 590.57                 Driver Version: 591.86          CUDA Version: 13.1     |
+-----+-----+
| GPU   Name                               Persistence-M  Bus-Id        Disp.A    Volatile Uncorr. ECC  |
| Fan  Temp   Perf         Pwr:Usage/Cap       Memory-Usage  GPU-Util  Compute M.  |
|                               MIG M.               |
+-----+-----+
|  0  NVIDIA GeForce RTX 5090              On          00000000:02:00.0  On          N/A          |
| 0%   47C    P8               27W / 575W        1473MiB / 32607MiB      0%         Default    |
|                               575W                    |
+-----+-----+
```

2. 사용환경설정

2-2. Linux 환경 세팅 상세 가이드

2-2. Linux 환경 - Nvidia Driver 설치

모든 **명령어**(빨간색으로 표시)는 Bash (Terminal)에 입력

대부분의 경우 이미 그래픽 드라이버가 설치되어 있음.

A-1. Bash (Terminal)에 입력

nvidia-smi

아래 이미지처럼 이미 드라이버가 존재하면(드라이버 버전이 뜨면) A과정은 생략하고 2번 과정부터 진행. 그렇지 않은 경우 -> [A-2](#). (다음페이지)

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ nvidia-smi
Thu Apr 23 15:58:37 2026

+-----+
| NVIDIA-SMI 570.211.01                Driver Version: 570.211.01   CUDA Version: 12.8   |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                               |                      | MIG M. |
+-----+-----+
|  0  NVIDIA GeForce RTX 5090               Off | 00000000:02:00.0  On |          N/A |
|  0%   47C   P8              42W /  575W |  598MiB / 32607MiB |      3%    Default |
|                               |                      | N/A |
+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
|      ID   ID              |                 |           Usage |
+-----+-----+
|  0   N/A  N/A         1825     G   /usr/lib/xorg/Xorg                  229MiB |
|  0   N/A  N/A         2011     G   /usr/bin/gnome-shell                 56MiB |
|  0   N/A  N/A         2612     G   ...exec/xdg-desktop-portal-gnome     8MiB |
|  0   N/A  N/A         2836     G   /usr/bin/nautilus                   18MiB |
|  0   N/A  N/A         3966     G   .../7766/usr/lib/firefox/firefox    185MiB |
+-----+-----+
```

2-2. Linux 환경 - Nvidia Driver 설치

A-2. 기존 드라이버 완전 제거 (기존 드라이버가 조금이라도 남아있는 게 존재하면 버전 충돌이 발생할 수 있기 때문.)

```
sudo apt remove --purge '^nvidia-.*'
```

```
sudo apt autoremove --purge -y
```

```
sudo reboot
```

A-3. 재부팅 후 추천 드라이버 확인

```
ubuntu-drivers devices
```

```
user@user-Z890-AORUS-ELITE-WIFI7: ~  
user@user-Z890-AORUS-ELITE-WIFI7:~$ ubuntu-drivers devices  
vendor   : NVIDIA Corporation  
driver   : nvidia-driver-590-server - distro non-free  
driver   : nvidia-driver-590-open  - distro non-free  
driver   : nvidia-driver-570-open  - distro non-free recommended  
driver   : nvidia-driver-570-server - distro non-free  
driver   : nvidia-driver-580-server - distro non-free  
driver   : nvidia-driver-580       - distro non-free  
driver   : nvidia-driver-590       - distro non-free  
driver   : nvidia-driver-580-open  - distro non-free  
driver   : nvidia-driver-570-server-open - distro non-free  
driver   : nvidia-driver-570       - distro non-free  
driver   : nvidia-driver-580-server-open - distro non-free  
driver   : nvidia-driver-590-server-open - distro non-free  
driver   : xserver-xorg-video-nouveau - distro free builtin
```

A-4. Recommend Driver 설치

(우리의 예시에선 570-open 버전, 하드웨어마다 recommend가 다를 수 있으니 주의)

```
sudo apt update
```

```
sudo apt install nvidia-driver-570-open
```

```
sudo reboot
```

A-5. Secure Boot 확인

```
mokutil --sb-state
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ mokutil --sb-state  
SecureBoot disabled
```

SecureBoot가 Enabled인 상태이면 NVIDIA 드라이버가 로드 실패하는 경우가 매우 흔함.

Enabled 인 경우

- BIOS 진입
- Settings -> Miscellaneous -> Secure Boot -> Disabled 혹은 Boot -> Secure Boot -> Disabled로 변경
- Save & Exit

2-2. Linux 환경 - Nvidia Driver 설치

A-6. 최종 확인

`nvidia-smi`

오른쪽 이미지처럼 모두 뜨면 정상.

정상 출력이 아닌 경우

- [해당 페이지](#) 1-2.부터 다시 시도
- 사용자의 환경과의 호환성을 고려하여 Recommend driver보다 더 낮은 버전을 설치. (권장)

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ nvidia-smi
Thu Apr 23 15:58:37 2026

+-----+
| NVIDIA-SMI 570.211.01                  Driver Version: 570.211.01      CUDA Version: 12.8     |
+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                           | MIG M.         |                      |
+-----+-----+
|  0  NVIDIA GeForce RTX 5090               Off | 00000000:02:00.0 On |          N/A         |
|  0%   47C   P8              42W /  575W |  598MiB / 32607MiB |      3%      Default |
|                                           |                      | N/A                  |
+-----+-----+

+-----+
| Processes:                              |
| GPU   GI    CI          PID    Type   Process name                  GPU Memory |
|          ID    ID                                   |            Usage   |
+-----+-----+
|  0     N/A   N/A         1825     G   /usr/lib/xorg/Xorg             229MiB |
|  0     N/A   N/A         2011     G   /usr/bin/gnome-shell           56MiB |
|  0     N/A   N/A         2612     G   ...exec/xdg-desktop-portal-gnome 8MiB  |
|  0     N/A   N/A         2836     G   /usr/bin/nautilus              18MiB |
|  0     N/A   N/A         3966     G   .../7766/usr/lib/firefox/firefox 185MiB |
+-----+-----+
```

2-2. Linux 환경 - Docker 설치

B-1. 패키지 업데이트

`sudo apt update`

`sudo apt install ca-certificates curl gnupg`

```
user@user-Z890-AORUS-ELITE-WIFI7: ~  
+-----+  
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo apt update  
sudo apt install ca-certificates curl gnupg  
[sudo] password for user:
```

B-2. Docker GPG Key 등록

`sudo install -m 0755 -d /etc/apt/keyrings`

`curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \`

`sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg`

`sudo chmod a+r /etc/apt/keyrings/docker.gpg`

```
user@user-Z890-AORUS-ELITE-WIFI7: ~  
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo install -m 0755 -d /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

B-3. Docker Repository 등록

`echo \`

`"deb [arch=$(dpkg --print-architecture) \`

`signed-by=/etc/apt/keyrings/docker.gpg] \`

`https://download.docker.com/linux/ubuntu \`

`$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \`

`sudo tee /etc/apt/sources.list.d/docker.list > /dev/null`

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ echo \  
"deb [arch=$(dpkg --print-architecture) \  
signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

2-2. Linux 환경 - Docker 설치

B-4. Docker 설치

`sudo apt update`

`sudo apt install docker-ce docker-ce-cli containerd.io` ₩

`docker-buildx-plugin docker-compose-plugin`

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo apt update
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo apt install docker-ce docker-ce-cli containerd.io \
docker-buildx-plugin docker-compose-plugin
```

B-5. Docker 서비스 시작

`sudo systemctl start docker`

B-6. 부팅 시 자동 실행

`sudo systemctl enable docker`

B-7. sudo 없이 Docker 사용

`sudo usermod -aG docker $USER`

`newgrp docker`

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo systemctl start docker
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo systemctl enable docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd
-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo usermod -aG docker $USER
user@user-Z890-AORUS-ELITE-WIFI7:~$ newgrp docker
```

B-8. 재부팅 - 컴퓨터 종료 후 다시 시작.

B-9. Docker 정상 작동 확인

`docker --version`

`docker compose version`

`docker ps`

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ docker --version
Docker version 29.4.1, build 055a478
user@user-Z890-AORUS-ELITE-WIFI7:~$ docker compose version
Docker Compose version v5.1.3
user@user-Z890-AORUS-ELITE-WIFI7:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

오른쪽 이미지처럼 모두 뜨면 정상.

2-2. Linux 환경 - Nvidia Container Toolkit 설치

C-1. GPG Key 등록

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey | \
sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ curl -fsSL https://nvidia.github.io/libnvidia-container/
gpgkey | \
sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-keyring.gpg
```

C-2. Repository 등록

```
curl -s -L https://nvidia.github.io/libnvidia-container/stable/deb/nvidia-container-toolkit.list | \
sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg] https://#g' | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ curl -s -L https://nvidia.github.io/libnvidia-container/
stable/deb/nvidia-container-toolkit.list | \
sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-container-toolkit-keyring.gpg]
https://#g' | \
sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

C-3. Toolkit 설치

```
sudo apt update
sudo apt install -y nvidia-container-toolkit
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo apt update
sudo apt install -y nvidia-container-toolkit
```

C-4. Docker Runtime 연결

```
sudo nvidia-ctk runtime configure --runtime=docker
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo nvidia-ctk runtime configure --runtime=docker
INFO[0000] Config file does not exist; using empty config
INFO[0000] Wrote updated config to /etc/docker/daemon.json
INFO[0000] It is recommended that docker daemon be restarted.
```

C-5. Docker 재시작

```
sudo systemctl restart docker
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ sudo systemctl restart docker
```

2-2. Linux 환경 - 최종 테스트

D-1. 최종 확인 명령어

`nvidia-smi`

`docker --version`

`docker compose version`

`docker ps`

정상출력

```
user@user-Z890-AORUS-ELITE-WIFI7:~$ nvidia-smi
Thu Apr 23 15:58:37 2026

+-----+
| NVIDIA-SMI 570.211.01                  Driver Version: 570.211.01      CUDA Version: 12.8     |
+-----+-----+-----+
| GPU   Name                               Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan   Temp   Perf              Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|                                       |                    |            MIG M. |
+-----+-----+-----+
| 0  NVIDIA GeForce RTX 5090               Off | 00000000:02:00.0  On |          N/A |
| 0%    47C    P8              42W / 575W | 598MiB / 32607MiB |      3%    Default |
|                                       |                    |            N/A |
+-----+-----+-----+

+-----+
| Processes: |
| GPU   GI   CI          PID    Type   Process name                      GPU Memory |
|          ID   ID                  |                 | Usage |
+-----+-----+-----+
| 0  N/A  N/A             1825     G   /usr/lib/xorg/Xorg                  229MiB |
| 0  N/A  N/A             2011     G   /usr/bin/gnome-shell                56MiB |
| 0  N/A  N/A             2612     G   ...exec/xdg-desktop-portal-gnome     8MiB |
| 0  N/A  N/A             2836     G   /usr/bin/nautilus                   18MiB |
| 0  N/A  N/A             3966     G   .../7766/usr/lib/firefox/firefox    185MiB |
+-----+-----+-----+

user@user-Z890-AORUS-ELITE-WIFI7:~$ docker --version
Docker version 29.4.1, build 055a478

user@user-Z890-AORUS-ELITE-WIFI7:~$ docker compose version
Docker Compose version v5.1.3

user@user-Z890-AORUS-ELITE-WIFI7:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS        NAMES
```

데이터 어노테이션 워크플로우

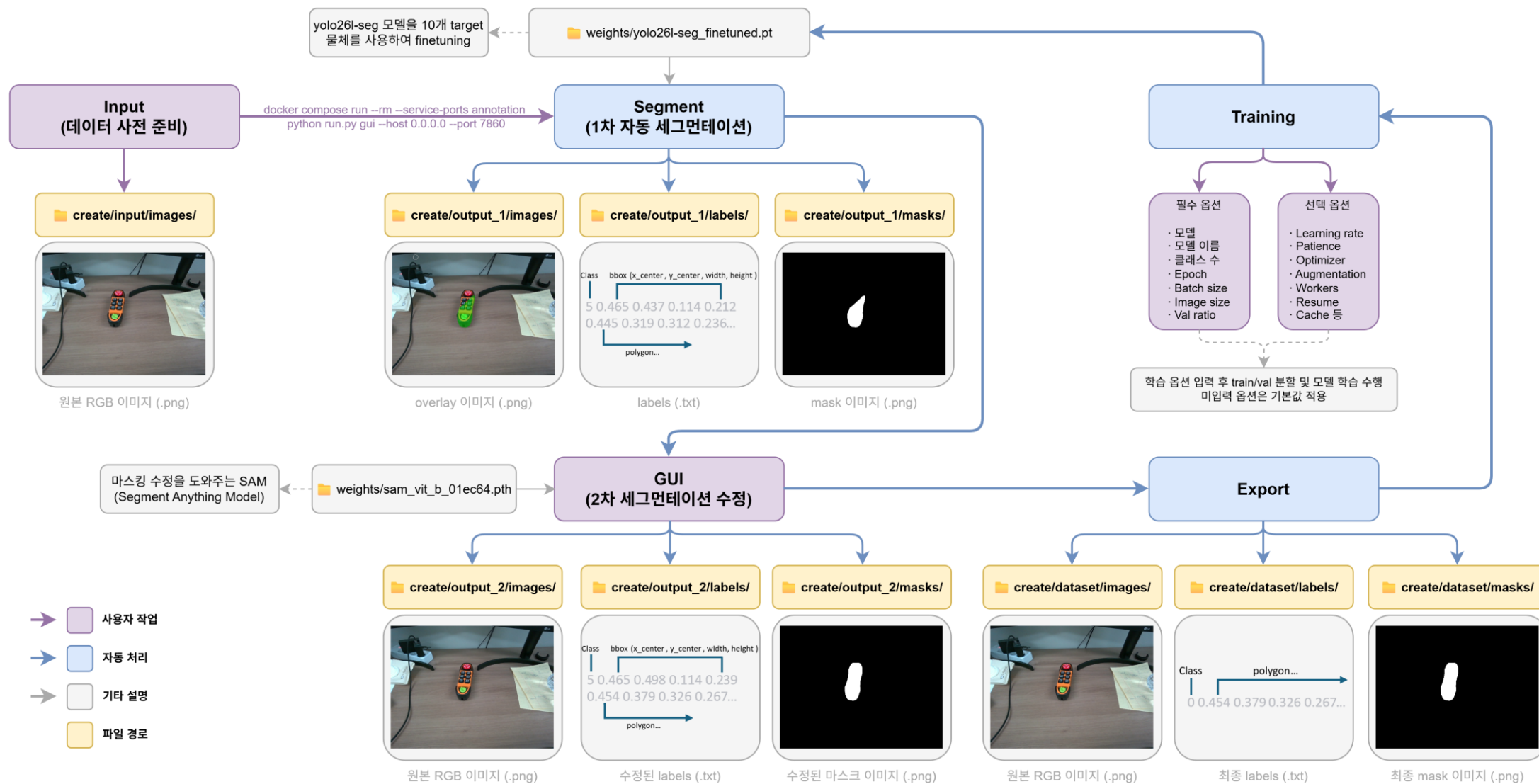
[단계별 실행 방법]

[윈도우 리눅스 공통]

3-1. 데이터 어노테이션 워크플로우 Overview

YOLO26l-seg 기반 Image Segmentation 및 마스크 수정 프로그램

- 640×480 PNG 이미지에서 자동 segmentation 및 수동 수정 후 최종 데이터셋 생성



3-2. 도커 이미지 빌드

도커 이미지 빌드 - 최초 1회만 빌드

- `docker compose build`
- 프로젝트 경로(Dockerfile이 있는 곳) 터미널에서 명령 입력
- 도커 이미지 빌드에 약 25분 내외 소요 (100Mbps 기준, 시스템 및 네트워크 환경에 따라 상이)

```
PS C:\Users\user\Desktop\pj\research\data_annotation_sw> docker compose build
```

```
user@user-Z890-AORUS-ELITE-WIFI7:~/Downloads/data_annotation_sw$ docker compose build
```

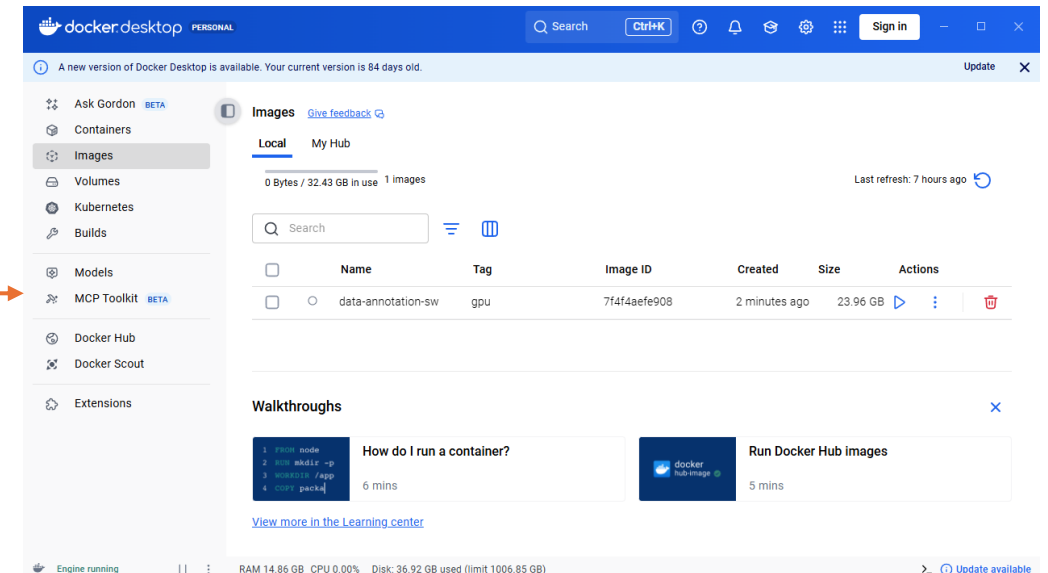
오류 (예외 사항) - Linux

```
permission denied while trying to connect to the docker API at unix:///var/run/docker.sock
```

- 위 오류의 경우
- `sudo docker compose build`
- 위 명령어로 실행 (권한 문제)

오류 (예외 사항) - Windows

- error during connect: dockerDesktopLinuxEngine...
- failed to connect to the docker API at npipe:...
- Docker Desktop이 실행되지 않았을 때의 오류
- Docker Desktop 실행



3-3. 프로그램 실행

프로그램 실행

- `docker compose run --rm --service-ports annotation python run.py gui --host 0.0.0.0 --port 7860`
- 프로젝트 경로(Dockerfile이 있는 곳) 터미널에서 명령 입력

```
\data_annotation_sw> docker compose run --rm --service-ports annotation python run.py gui --host 0.0.0.0 --port 7860
```

실행 방식 설명

- Docker Container 내부에서 GUI 서버가 실행되는 형태
- 7860 포트를 통해 웹 기반 GUI 접속이 가능함

접속 주소

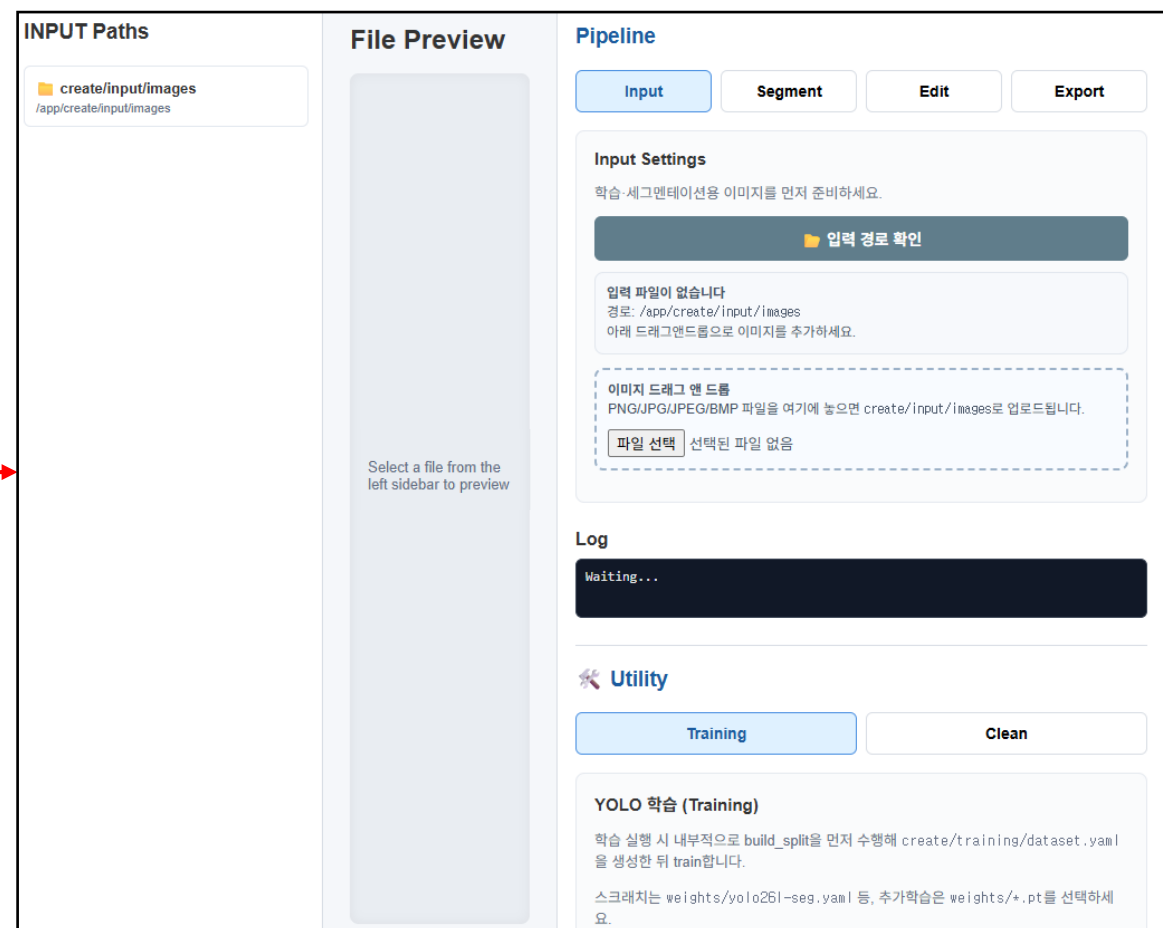
- <http://localhost:7860>

기능

- Pipeline - Input: 이미지 업로드
- Pipeline - Segment: 원본 이미지에 대해 1차 세그멘테이션 수행
- Pipeline - Edit: 생성된 마스크 영역 수정 및 검수
- Pipeline - Export: 수정 결과를 최종 데이터셋으로 구성
- Utility - Training: 생성된 데이터셋으로 모델 학습(Fine-tuning / Retraining)을 수행
- Utility - Clean: 작업 폴더 정리

참고 사항

- Docker Desktop 또는 Docker Engine이 실행 중이어야 함



3-4. 이미지 업로드 - Input

PNG 이미지 업로드

- GUI 기반으로 PNG 이미지를 직접 업로드하여 세그멘테이션 작업에 사용할 입력 데이터를 생성

업로드 방식

- Input 탭에서 Drag & Drop 또는 파일 선택 버튼을 통해 이미지 업로드
- 파일 선택: 파일을 선택하여 create/input/images 경로에 업로드
- 직접 업로드: 파일을 create/input/images 경로에 직접 업로드도 가능

지원 형식

- PNG / JPG / JPEG / BMP

저장 경로

- 업로드된 이미지는 create/input/images 경로에 저장

주요 기능

- 여러 장의 이미지를 일괄 업로드 가능
- 파일명에서 첫 언더바(_) 앞의 값이 클래스 번호가 아닌 경우 class_number 직접 입력
- 입력한 class_number를 업로드 이미지들에 일괄 적용
- 업로드 후 Segment 탭에서 자동 세그멘테이션 수행

Pipeline

Input

Segment

Edit

Export

Input Settings

학습·세그멘테이션용 이미지를 먼저 준비하세요.

입력 경로 확인

입력 파일이 없습니다

경로: /app/create/input/images

아래 드래그앤드롭으로 이미지를 추가하세요.

이미지 드래그 앤 드롭

PNG/JPG/JPEG/BMP 파일을 여기에 놓으면 create/input/images로 업로드됩니다.

파일 선택

선택된 파일 없음

입력 확인됨

경로: /app/create/input/images

파일 수: 469

localhost:7860 내용:

3개 파일의 파일명이 클래스 번호로 시작하지 않습니다.

전체에 적용할 class_number를 입력하세요.

첫 파일: test1.png

4

확인

취소

3-5. 1차 자동 세그멘테이션 - Segment

Yolo 기반 자동 Segmentation

- 업로드한 이미지로 자동 segmentation 수행
- 입력 이미지에 대해 객체 검출 + 마스크 생성 자동 수행
- 이후 GUI 정제를 위한 초기 마스크 생성

사용 모델

- 모델 변경 가능 (기본 모델 경로는 10가지 객체에 대해 파인튜닝된 모델)
- YOLO_WEIGHT = WEIGHTS_DIR / "yolo26l-seg_finetuned.pt"

생성 결과

저장 위치

- create/output_1/

생성 파일

- Images – 시각화 이미지 (images/, overlay)
- Labels – 라벨 파일 (labels/, txt) <클래스ID, bbox, 마스크 좌표>
- masks – 마스크 이미지 (masks/)



<시각화 이미지>



<마스크 이미지>

Class	bbox (x_center, y_center, width, height)
0	0.465411 0.498339 0.114340 0.238739
0.454688	0.379167...

→ polygon...

<라벨 파일>

Pipeline

Input

Segment

Edit

Export

Segment Settings

Segmentation Weights (.pt)

기본값 사용 (config.py YOLO_WEIGHT) ▼

기본값 사용 (config.py YOLO_WEIGHT)

yolo26l-seg.pt

yolo26l-seg_finetuned.pt

▶ Run Segment

Log

```
[running] python run.py segment

WARNING ⚠ user config directory '/tmp/Ultralytics/Ultralytics' is not
writable, using '/tmp/Ultralytics'. Set YOLO_CONFIG_DIR to override.
Creating new Ultralytics Settings v0.0.6 file ✅
View Ultralytics Settings with 'yolo settings' or at
'/tmp/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings
runs_dir=path/to/dir'. For help see
https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Device: cuda
Loading YOLO-seg...
Total images: 469
YOLO batches: [#####-----] 63% (19/30) [00:12<00:07, 1.57it/s]
```

3-6. 2차 세그멘테이션 수정 - GUI

GUI 기반 마스크 정제

- 1차 생성된 마스크를 웹 GUI에서 수정
- 이후 GUI 정제를 위한 초기 마스크 생성 단계

Pipeline



GUI 주요 기능

- Brush: 마스크를 직접 칠해 추가 (단축키: 1)
- Eraser: 기존 마스크를 지워 제거 (단축키: 2)
- SAM Point: 클릭 위치 기반으로 자동 마스크 생성 (좌클릭: positive, 우클릭: negative, point 누적 반영) (단축키: 3)
- Brush Size: 브러시/지우개 크기 조절
- Undo: 직전 편집 되돌리기 (단축키: ctrl + z)
- Erase All: 현재 마스크 전체 삭제
- Zoom: 확대/축소로 세밀한 편집 수행
- Prev / Next: 이전 이미지 / 다음 이미지 이동 (단축키: A / D) -> 이동 시 마스크 변화가 존재하면 자동 저장
- Save: 현재 편집 결과 저장 (단축키: ctrl + s)

생성 결과

저장 위치

- create/output_2/

생성 파일

- images – 원본 이미지 (images/) <학습/테스트 사용 편의를 위해 원본 이미지로 저장>
- labels – 수정된 라벨 파일 (labels/, txt) <클래스ID, bbox, 마스크 좌표>
- masks – 수정된 마스크 이미지 (masks/)

3-6. 2차 세그멘테이션 수정 - GUI

Images

Total: 469

SAM Point Assist

좌클릭: Positive point + 즉시 SAM 실행
우클릭: Negative point + 즉시 SAM 실행
클릭하는 기존 마스크를 덮어쓰고, 이후
Brush/Eraser로 수정 가능

1_1

1_2

1_3

1_4

1_5

1_6

1_7

1_8

1_9

1_10

1_11

1_12

1_13

1_14

1_15

1_16

1_17

1_18

1_19

1_20

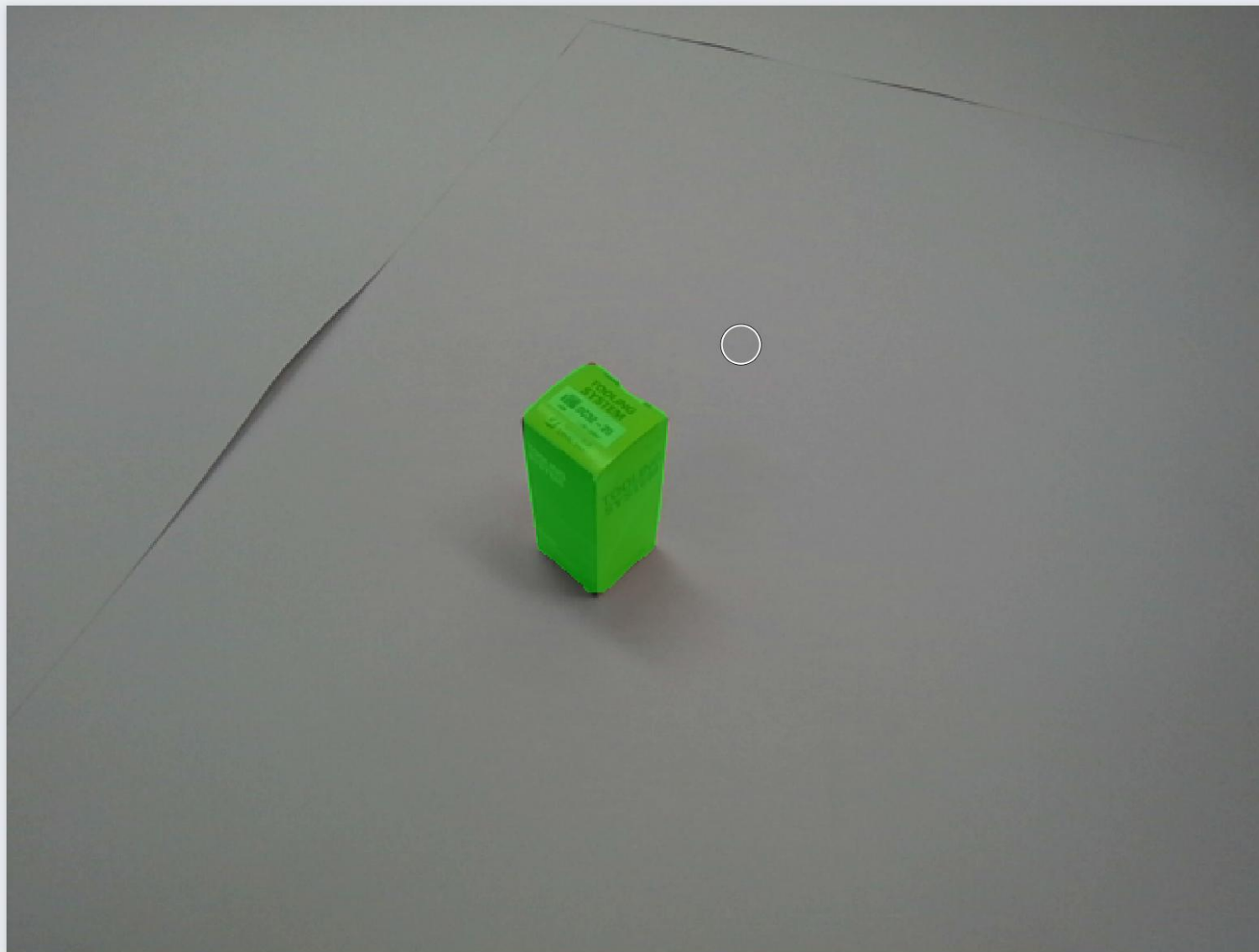
1_21

1_22

1_23

1_24

1_25



Brush

Eraser

SAM Point

Brush Size

Undo

Erase All

Zoom

Loaded: 1_1

◀ Prev

Next ▶

Save

1_1 | 640×480 | class 1

Pipeline

Input

Segment

Edit

Export

Log

```
[done] python run.py segment

WARNING ⚠ user config directory '/tmp/Ultralytics/Ultralytics' is not
writable, using '/tmp/Ultralytics'. Set YOLO_CONFIG_DIR to override.
Creating new Ultralytics Settings v0.0.6 file
View Ultralytics Settings with 'yolo settings' or at
/tmp/Ultralytics/settings.json
Update Settings with 'yolo settings key=value', i.e. 'yolo settings
runs_dir=path/to/dir'. For help see
https://docs.ultralytics.com/quickstart/#ultralytics-settings.
Device: cuda
Loading YOLO-seg...
Total images: 469
SUMMARY
Detected: 469
Saved: 469
YOLO batches: [#####] 100% (30/30) [00:19:00:00,
1.54it/s]
```

추가 유틸리티

Training

Clean

YOLO 학습 (Training)

학습 실행 시 내부적으로 build_split을 먼저 수행해 create/training/dataset.yaml을 생성한 뒤 train합니다.

스크래치는 weights/yolo26l-seg.yaml 등, 추가학습은 weights/*.pt를 선택하세요.

필수 옵션

기본 모델 (weights/*.pt 또는 *.yaml)

yolo26l-seg.pt

모델 이름 (exp_name)

예: my_exp

클래스 개수 (num_classes)

10

Epoch

Batch size

Image size

Val ratio

50

16

640

0.2

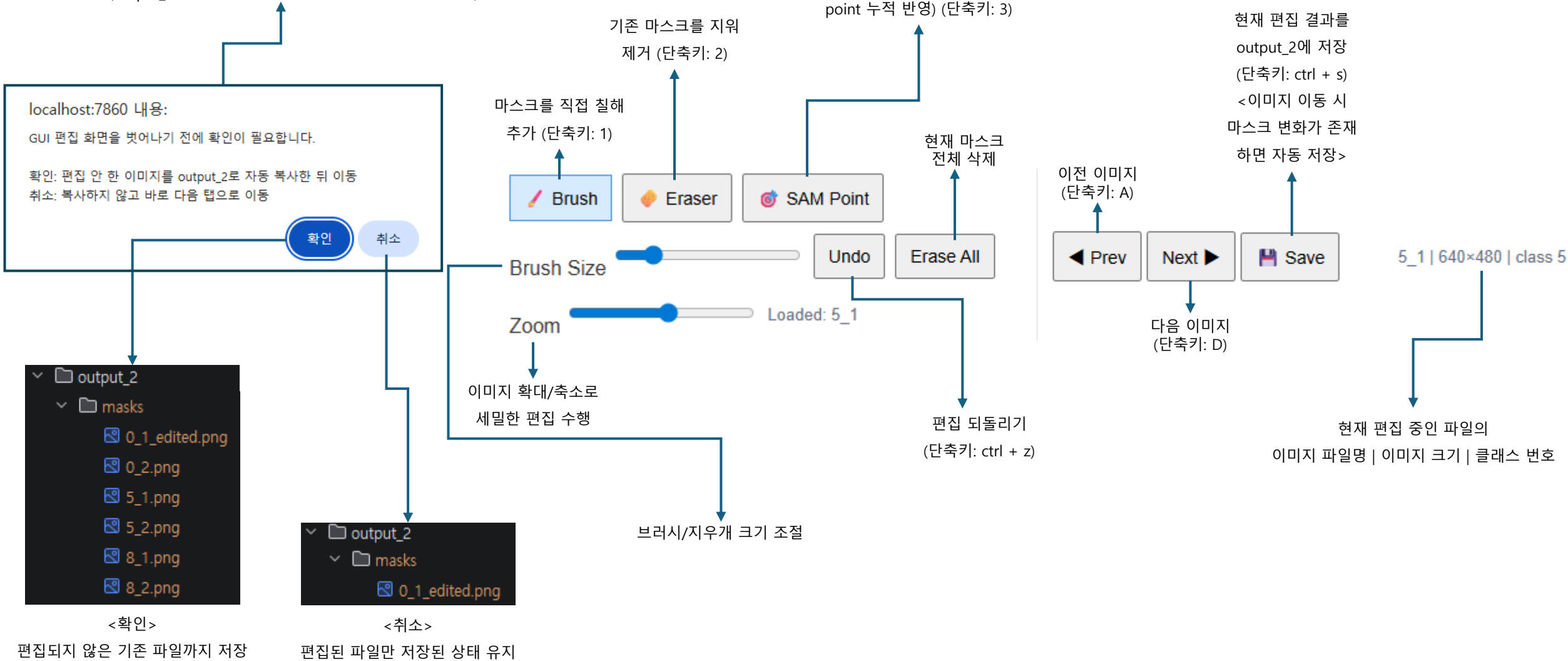
선택 옵션

Learning rate

Patience

3-6. 2차 세그멘테이션 수정 - GUI

수정하지 않은 이미지 output_1 결과를 그대로 output_2로 복사하여 최종본으로 넘김
(output_2에 edited 파일이 존재하면 덮어 씌우지 않음)



3-7. 데이터 내보내기 - Export

Dataset 추출

- GUI에서 수정 및 검수한 세그멘테이션 결과를 최종 학습용 Dataset 구조로 변환 및 저장

실행 방식

- Export 탭을 통해 Dataset 생성 수행
- 수정 완료된 이미지, 라벨, 마스크 데이터를 통합하여 저장
- 파일명 앞에 배경/환경 등 접두사를 추가하여 다양한 촬영 환경의 데이터를 구분 및 관리

생성 결과

저장 위치

- create/dataset/

생성 파일

- images – RGB 이미지 (images/)
- labels – 라벨 파일 (labels/, txt) <클래스ID, 마스크 좌표>
- masks – 마스크 이미지 (masks/)

Pipeline

Input

Segment

Edit

Export

Export Settings

Export Prefix (bg)

Environment

▶ Run Export

Log

```
[done] python run.py export --bg Environment

Total masks: 469
Background prefix: Environment
Exported: 469
Dataset: /app/create/dataset
```

3-8. 학습 - Training

YOLO 학습 (Training)

- 생성된 Dataset(📁 create/dataset/)을 기반으로 YOLO26l-seg 모델 학습 수행

실행 방식

- Training 탭에서 학습 옵션 설정 후 학습 실행
- 내부적으로 build_split을 먼저 수행하여 Train / Validation Dataset 자동 분할 후 학습 진행
- create/training/dataset.yaml 파일 생성 후 YOLO 학습 환경 자동 구성

학습 방식

- Scratch 학습
- yolov26l-seg.yaml 기반 신규 학습
- 추가 학습(Fine-tuning) => 기존 학습된 가중치가 있는 모델은 📁 weight/ 경로에 있음
- 기존 weights/*.pt 모델 기반 추가 학습 가능

필수 옵션

- 기존 모델 - YOLO 기본 모델 또는 사전 학습 모델 선택
- exp_name - 학습 결과 저장 폴더 이름 지정
- num_classes - 학습 클래스 개수 설정
- Epoch - 전체 학습 반복 횟수
- Batch size - 한 번에 학습할 이미지 개수
- Image size - 입력 이미지 크기 설정
- Val ratio - Validation Dataset 비율 설정

선택 옵션

- Learning rate
 - Mixed precision(fp16)
 - Optimizer
 - Cosine LR
 - Augmentation
 - Weight decay
 - Workers
 - Dropout
 - Seed
 - Cache
 - Freeze layers
 - Resume checkpoint 등 추가 학습 옵션 지원
- 선택 옵션 미설정 시 YOLO26l-seg 기본 학습 설정(Default Hyperparameter) 기반으로 자동 학습 수행

YOLO 학습 (Training)

학습 실행 시 내부적으로 build_split을 먼저 수행해 create/training/dataset.yaml 을 생성한 뒤 train합니다.

스크래치는 weights/yolo26l-seg.yaml 등, 추가학습은 weights/*.pt를 선택하세요.

필수 옵션

기존 모델 (weights/*.pt 또는 *.yaml)

yolo26l-seg.pt

모델 이름 (exp_name)

예: my_exp

클래스 개수 (num_classes)

10

Epoch

50

Batch size

16

Image size

640

Val ratio

0.2

선택 옵션

Learning rate

기본

Patience

기본

Optimizer

auto

Freeze layers

0=off

Augmentation

medium

Workers

기본

Seed

기본

Resume checkpoint

체크포인트 경로 또는 비움

Resume auto

☐

Cache

(기본)

Mixed precision (fp16)

(기본)

Cosine LR

(기본)

Pretrained backbone

☐

Weight decay

기본

Dropout

기본

Save period

기본

▶ 학습 실행

■ 학습 중단

3-8. 학습 - Training

생성 결과

학습 결과 저장 위치

- weights/test(설정된 모델 이름)/

생성 파일

- results.csv
 - Epoch별 학습 결과 및 성능 로그 저장
 - Loss, mAP, Precision, Recall 등 기록
- args.yaml
 - 학습에 사용된 옵션 및 하이퍼파라미터 저장
- labels.jpg
 - Dataset 클래스 정보 및 라벨 분포 시각화 결과
- train_batch*.jpg
 - 학습 중 사용된 이미지 Batch 미리보기
 - Augmentation 및 라벨 적용 상태 확인 가능
- 기타 그래프 및 매트릭스
- best.pt
 - 최고 성능 가중치
- last.pt
 - 마지막 Epoch 가중치

이름	수정된 날짜	유형	크기
test	2026-05-19 오후 5:20	파일 폴더	
.gitkeep	2026-04-06 오후 12:09	GITKEEP 파일	0KB
sam_vit_b_01ec64.pth	2026-02-19 오후 5:59	PTH 파일	366,253KB
yolo26l-seg.pt	2026-05-12 오후 2:58	PT 파일	62,208KB
yolo26l-seg_finetuned.pt	2026-04-06 오전 9:44	PT 파일	61,961KB



이름	수정된 날짜	유형	크기
best.pt	2026-05-19 오후 5:21	PT 파일	185,194KB
last.pt	2026-05-19 오후 5:21	PT 파일	185,194KB

3-9. 기타 기능

작업 폴더 정리 (Clean)

- 생성된 작업 결과 및 중간 데이터를 선택적으로 초기화하는 기능

실행 방식

- Clean 탭에서 삭제 대상을 선택 후 선택 폴더 초기화 버튼을 통해 작업 폴더 정리 수행

삭제 대상

- all - 전체 작업 폴더 초기화
- input - 업로드한 입력 이미지 삭제
- output1 - 자동 세그멘테이션 결과 삭제
- output2 - GUI 수정 결과 삭제
- dataset - 최종 생성 Dataset 삭제
- training - 학습 결과 및 가중치 삭제

주의 사항

- 삭제된 데이터는 복구되지 않음
- 필요한 중간, 최종 결과는 별도 백업 후 사용 권장



Utility

Training

Clean

작업 폴더 정리 (Clean)

삭제 대상

all - 전체 작업 폴더 초기화

all - 전체 작업 폴더 초기화

input - 업로드한 입력 이미지 삭제

output1 - 자동 세그멘테이션 결과 삭제

output2 - Edit 수정 결과 삭제

dataset - 최종 생성 Dataset 삭제

training - 학습 결과 및 가중치 삭제



Utility

Training

Clean

작업 폴더 정리 (Clean)

삭제 대상

all - 전체 작업 폴더 초기화

선택 폴더 초기화